

PROYECTO DE INTELIGENCIA ARTIFICIAL

Algoritmo de Búsqueda A* para Rescate de Emergencia Post-Terremoto

$$f(n) = g(n) + h(n)$$

Informe técnico del proyecto

Integrantes

Integrante 1 – (nombre completo)

Integrante 2 – (nombre completo)

Integrante 3 – (nombre completo)

Integrante 4 – (nombre completo)

Asignatura: Inteligencia Artificial

Docente: (nombre del docente)

Lenguaje: Python 3

Fecha: 28/09/2026

Contenido

- 1. Descripción general del proyecto
- 2. Definición del entorno y pesos de la matriz
- 3. Modelado del problema de búsqueda
- 4. Flujo de ejecución del algoritmo A* (Pasos 1 a 4)
- 5. La heurística: por qué funciona y por qué es admisible
- 6. Ejecución paso a paso: lista abierta, lista cerrada, nodos creados y expandidos
- 7. Solución: ruta óptima y espacio de búsqueda
- 8. Implementación en Python y visualización
- 9. Conclusiones
- Anexo A. Traza completa de la lista abierta y la lista cerrada

1. Descripción general del proyecto

Este proyecto implementa el algoritmo de búsqueda informada **A*** (**A-Estrella**) para simular la navegación óptima de un bombero dentro de un entorno de desastre de **8 × 8** casilleros. Tras un evento sísmico de gran magnitud, la infraestructura de una edificación colapsa parcialmente, creando un escenario crítico donde el tiempo de respuesta es vital para rescatar a un superviviente atrapado.

El objetivo es encontrar la ruta de **menor coste total** desde la posición del Bombero **B (0,0)** hasta el Superviviente **S (7,7)**, evitando las casillas bloqueadas y decidiendo cuándo conviene rodear o atravesar las zonas con impedimentos. A* evalúa en cada iteración la función de coste total:

$$f(n) = g(n) + h(n)$$

- **g(n) (Coste Real)**: coste acumulado desde la casilla inicial (posición del bombero) hasta la casilla actual n .
- **h(n) (Heurística)**: estimación del coste restante desde la casilla actual n hasta la meta (superviviente). Se utiliza la **Distancia de Manhattan** dadas las restricciones del movimiento en cuadrícula (4 direcciones):

$$h(n) = |x_n - x_{\text{meta}}| + |y_n - y_{\text{meta}}|$$

El trabajo cubre los cuatro puntos de la rúbrica: (1) este informe explicando cada paso y la admisibilidad de la heurística; (2) la solución con representación gráfica y la visualización del espacio de búsqueda; (3) la visualización de las variables importantes (lista cerrada, lista abierta, nodos creados y expandidos) y (4) el material de apoyo para el video explicativo.

2. Definición del entorno y pesos de la matriz

El escenario se modela mediante una matriz de **8 × 8** casilleros donde el terreno presenta distintos grados de dificultad y accesibilidad:

Tipo de casilla	Coste	Descripción
Libre / Transitado	1	Terreno despejado y seguro para avanzar a velocidad normal.
Paso Prohibido / Infranqueable (X)	∞	Paredes colapsadas, incendios totales o fallas estructurales. El bombero no puede transitar.
Impedimento Leve – Escombros Menores	2	Residuos ligeros de concreto o mobiliario caído. Reduce levemente el avance.
Impedimento Moderado – Grietas / Humo Denso	4	Daños estructurales moderados o baja visibilidad que requieren precaución.
Impedimento Alto – Fuego Parcial / Agua Acumulada	7	Zonas peligrosas que exigen un esfuerzo y tiempo considerable para ser atravesadas.

Matriz de terreno y ponderación de costes (la del enunciado, con B y S de valor 1 según la aclaración del profesor):

F \ C	0	1	2	3	4	5	6	7
0	B (1)	1	2	4	X	1	1	1
1	1	X	1	7	X	2	X	1
2	2	X	1	1	1	4	X	1
3	1	2	2	X	X	1	1	2
4	4	X	1	7	1	2	X	4
5	1	X	1	1	X	1	1	1
6	2	1	4	1	X	2	X	1
7	1	1	7	2	1	1	1	S (1)

B: Bombero (0,0), valor 1 · S: Superviviente (7,7), valor 1 · X: Bloqueado (∞) · 1: Libre · 2: Escombros leves · 4: Grietas / Humo · 7: Fuego parcial / Agua acumulada

Convención de coste de movimiento. Moverse de una casilla a una vecina cuesta el **peso de la casilla a la que se entra**. Así, $g(n)$ acumula los pesos de todas las casillas atravesadas después de la inicial. Por aclaración del profesor, la casilla del Bombero B (0,0) y la del Superviviente S (7,7) se toman como puntos de **valor 1** (B (1) y S (1)): conservan su identificación como B y S, pero cuestan lo mismo que una casilla libre. Esto no altera la solución, porque los valores altos (2, 4 y 7) se usan como penalización para evitar determinados tránsitos, y cualquier ruta debe entrar a S exactamente una vez. Como el algoritmo parte de B con $g(0,0) = 0$, el valor de B solo se usaría si se volviera a entrar en ella, lo cual nunca ocurre porque queda en la lista cerrada. Las casillas X tienen coste ∞ y nunca se generan como sucesores.

3. Modelado del problema de búsqueda

Para aplicar A* el escenario se formaliza como un problema de búsqueda en un grafo:

Elemento	Definición en este proyecto
Estado	Posición (F, C) del bombero en la matriz, con $0 \leq F, C \leq 7$.
Estado inicial	(0,0) – casilla del Bombero, con $g(0,0) = 0$.
Acciones	Moverse a una de las 4 casillas contiguas, en este orden: arriba (F-1), abajo (F+1), izquierda (C-1), derecha (C+1). Sin movimientos diagonales.
Restricciones	No se puede salir del tablero ni entrar en casillas X (coste ∞).
Coste de paso $c(n, n')$	Peso de la casilla destino n' : 1, 2, 4 o 7 (B y S valen 1).
Test de meta	La casilla (7,7) (Superviviente) es extraída de la lista abierta.
Coste del camino	Suma de los costes de paso: $g(\text{meta})$.
Heurística	$h(n) = F_n - 7 + C_n - 7 $ (Distancia de Manhattan).

Definiciones de las variables que se visualizan:

- **Lista abierta (frontera):** nodos generados pendientes de evaluar. De ella se extrae siempre el nodo de menor $f(n)$.
- **Lista cerrada:** nodos ya extraídos y evaluados; no se vuelven a procesar.
- **Nodo creado (generado):** cada vez que una casilla se inserta por primera vez en la lista abierta (incluye el nodo inicial).

- **Nodo expandido:** nodo extraído de la lista abierta cuyos 4 vecinos se examinan. La meta se extrae pero no se expande, porque en ese momento termina la búsqueda.
- **Actualización:** cuando se descubre un camino más barato hacia un nodo que ya estaba en la lista abierta; se reduce su $g(n)$ y se cambia su padre.
- **Criterio de desempate:** si dos nodos tienen el mismo $f(n)$ se elige el de menor $h(n)$ (el que está más cerca de la meta) y, si persiste el empate, el que fue creado primero. Esto hace que la ejecución sea determinista y reproducible.

4. Flujo de ejecución del algoritmo A*

Paso 1: Inicialización. Se posiciona al Bombero en la casilla inicial (0,0) y al Superviviente en la casilla meta (7,7). Se inserta el nodo inicial en la lista abierta con $g(0,0) = 0$, $h(0,0) = 14$ y $f(0,0) = 14$. La lista cerrada comienza vacía.

Paso 2: Evaluación y Búsqueda. En cada iteración el algoritmo extrae el nodo con la menor función $f(n)$ de la lista abierta y lo pasa a la lista cerrada. Revisa sus 4 vecinos contiguos (arriba, abajo, izquierda, derecha) omitiendo casillas bloqueadas (X) o nodos ya evaluados (lista cerrada). Para cada vecino válido calcula $g = g(\text{actual}) + \text{coste de la casilla}$, h con Manhattan y $f = g + h$; si el vecino es nuevo lo inserta en la lista abierta, y si ya estaba pero se llegó con un g menor, actualiza su g y su padre.

Paso 3: Selección de Ruta Óptima. Como la lista abierta siempre entrega el nodo de menor f , el algoritmo balancea dinámicamente entre rodear zonas peligrosas usando vías libres (coste 1) o atravesar zonas de coste 2, 4 o 7 cuando la suma $g(n) + h(n)$ demuestra que es la vía más barata en coste total acumulado.

Paso 4: Criterio de Parada. El proceso concluye con éxito cuando el nodo meta (7,7) es extraído de la lista abierta. Entonces se reconstruye el camino óptimo siguiendo los punteros *padre* desde la meta hasta el inicio, y se invierte. Si la lista abierta se vaciara antes, no existiría camino.

Pseudocódigo implementado (función `a_estrella` del archivo `astar_rescate.py`):

```
ABIERTA ← { inicio con g=0, h=h(inicio), padre=nulo }
CERRADA ←
mientras ABIERTA no esté vacía:
    n ← nodo de ABIERTA con menor (f, h, orden de creación)
    mover n de ABIERTA a CERRADA
    si n = meta: devolver reconstruir_camino(n)
    para cada vecino v de n en (arriba, abajo, izquierda, derecha):
        si v es X o v en CERRADA: omitir
        g' ← g(n) + coste(v)
        si v en ABIERTA: crear v (g', h(v), padre=n) e insertarlo
        si no, si g' < g(v): g(v) ← g', padre(v) ← n
    devolver "no existe camino"
```

5. La heurística: por qué funciona y por qué es admisible

5.1 ¿Por qué funciona la Distancia de Manhattan?

El bombero solo puede moverse en 4 direcciones (arriba, abajo, izquierda, derecha). Cada movimiento cambia **exactamente una** coordenada en **exactamente una** unidad. Por lo tanto, para ir desde (F_n, C_n) hasta $(7,7)$ se necesitan, como mínimo, $|F_n - 7|$ movimientos verticales y $|C_n - 7|$ movimientos horizontales: la Distancia de Manhattan es el **número mínimo de pasos** hasta la meta en un tablero sin obstáculos. Esta información dirige la búsqueda: entre dos nodos con el mismo $g(n)$, A^* prefiere el que está más cerca del superviviente, en lugar de explorar a ciegas en todas las direcciones como haría una búsqueda de coste uniforme.

5.2 Demostración de admisibilidad

Una heurística es **admisble** si nunca sobreestima el coste real mínimo hasta la meta, es decir, si $h(n) \leq h^*(n)$ para todo nodo n , donde $h^*(n)$ es el coste del camino óptimo desde n hasta $(7,7)$.

Demostración. Sea $P = (n = n_0, n_1, \dots, n_k = \text{meta})$ cualquier camino válido desde n hasta la meta.

- Cada paso cambia una coordenada en una unidad, luego el valor de Manhattan cambia como máximo en 1 por paso: $|h(n_i) - h(n_{i+1})| = 1$. Como $h(\text{meta}) = 0$, el camino necesita $k \geq h(n)$ pasos.
- El coste de cada paso es el peso de la casilla a la que se entra, que pertenece al conjunto $\{1, 2, 4, 7\}$ (las X no se pueden pisar). Por tanto, $c(n_i, n_{i+1}) \geq 1$.
- Entonces $\text{coste}(P) = \sum c(n_i, n_{i+1}) \geq k \cdot 1 \geq h(n)$.

Como esto se cumple para cualquier camino P , también se cumple para el óptimo: **$h(n) \leq h^*(n)$** . Los obstáculos X solo pueden alargar el camino (nunca acortarlo), y los impedimentos 2, 4 y 7 solo pueden encarecerlo, así que la heurística sigue siendo una cota inferior. **La Distancia de Manhattan es admisible** en este problema. $\square$

5.3 Consistencia (monotonía)

Además es **consistente**: para todo nodo n y todo vecino n' se cumple $h(n) \leq c(n, n') + h(n')$. En efecto, $h(n) - h(n') \leq 1$ (se movió una casilla) y $c(n, n') \geq 1$. La consistencia implica que los valores de $f(n)$ nunca disminuyen a lo largo de un camino y que, cuando un nodo entra en la lista cerrada, ya se ha encontrado el camino más barato hasta él. Por eso es correcto **omitir los nodos de la lista cerrada** tal como pide el Paso 2 del enunciado: nunca será necesario reabrirlos.

5.4 Consecuencia: optimalidad garantizada

Con una heurística admisible, A^* es **óptimo**: cuando la meta sale de la lista abierta, $f(\text{meta}) = g(\text{meta}) + 0$ es menor o igual que el f de cualquier otro nodo pendiente, y como ningún f subestimado supera al coste real, ningún camino pendiente puede terminar con un coste menor. Por eso el Paso 4 detiene la búsqueda al **extraer** la meta y no al generarla.

5.5 Verificación experimental

El programa calcula $h^*(n)$ exacto para las 50 casillas transitables (búsqueda de coste uniforme desde cada una) y lo compara con $h(n)$. Resultado: **admisble = SÍ** ($h(n) \leq h^*(n)$ en las 50 casillas) y **consistente = SÍ** (se cumple la desigualdad triangular en todas las transiciones). La diferencia mínima $h^* - h$ fuera de la meta es 0, es decir, la heurística nunca sobreestima.

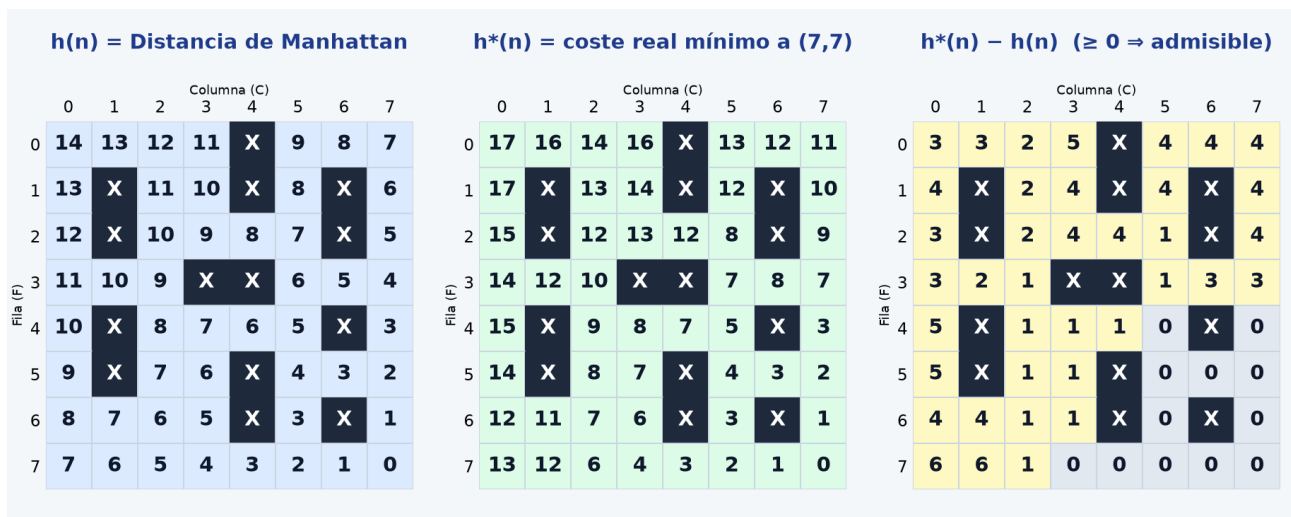


Figura 1. Izquierda: $h(n)$. Centro: coste real óptimo $h^*(n)$. Derecha: $h^*(n) - h(n) \geq 0$ en todas las casillas, lo que confirma la admisibilidad.

Observación. La diferencia $h^* - h$ va de 0 a 6. En 11 casillas es 0: allí la heurística coincide con el coste real, porque existe un camino directo hacia la meta formado solo por casillas de valor 1. En las demás, Manhattan subestima porque supone que no hay muros X ni casillas de coste 2, 4 o 7 que obliguen a rodear o a pagar más. Al estar tan cerca del coste real sin superarlo nunca, la heurística es admisible y además muy informada: guía la búsqueda casi directamente hacia el superviviente (ver Sección 7.3).

6. Ejecución paso a paso

La ejecución completa tuvo **21 iteraciones**. A continuación se detallan las primeras iteraciones con todos sus cálculos; la tabla resumen y el Anexo A contienen el resto. Las imágenes muestran el tablero en cada momento: casillas grises = lista cerrada (con su número de orden #k), recuadro azul discontinuo = lista abierta, recuadro magenta = nodo actual; en cada casilla conocida se indica g, h y f.

Iteración 0

Inicialización: se inserta (0,0) en la lista abierta con g=0, h=14, f=14.

Lista abierta (1): (0,0) [g=0, h=14, f=14]

Lista cerrada (0): (vacía) · **Creados:** 1 · **Expandidos:** 0

Iteración 1

Se expande (0,0) (f=14).

Dirección	Vecino	Coste	g	h	f	Acción
abajo	(1,0)	1	1	13	14	NUEVO - se inserta en lista abierta
derecha	(0,1)	1	1	13	14	NUEVO - se inserta en lista abierta

Lista abierta (2): (1,0) [g=1, h=13, f=14], (0,1) [g=1, h=13, f=14]

Lista cerrada (1): (0,0) · **Creados:** 3 · **Expandidos:** 1

Iteración 2

Se expande (1,0) (f=14).

Dirección	Vecino	Coste	g	h	f	Acción
arriba	(0,0)	1	–	–	–	En lista cerrada - se omite
abajo	(2,0)	2	3	12	15	NUEVO - se inserta en lista abierta
derecha	(1,1)	∞	–	–	–	BLOQUEADO (X) - se omite

Lista abierta (2): (0,1) [g=1, h=13, f=14], (2,0) [g=3, h=12, f=15]

Lista cerrada (2): (0,0) (1,0) · **Creados:** 4 · **Expandidos:** 2

Iteración 3

Se expande (0,1) (f=14).

Dirección	Vecino	Coste	g	h	f	Acción
abajo	(1,1)	∞	–	–	–	BLOQUEADO (X) - se omite
izquierda	(0,0)	1	–	–	–	En lista cerrada - se omite
derecha	(0,2)	2	3	12	15	NUEVO - se inserta en lista abierta

Lista abierta (2): (2,0) [g=3, h=12, f=15], (0,2) [g=3, h=12, f=15]

Lista cerrada (3): (0,0) (1,0) (0,1) · **Creados:** 5 · **Expandidos:** 3

Iteración 4

Se expande (2,0) (f=15).

Dirección	Vecino	Coste	g	h	f	Acción
arriba	(1,0)	1	—	—	—	En lista cerrada - se omite
abajo	(3,0)	1	4	11	15	NUEVO - se inserta en lista abierta
derecha	(2,1)	∞	—	—	—	BLOQUEADO (X) - se omite

Lista abierta (2): (3,0) [g=4, h=11, f=15], (0,2) [g=3, h=12, f=15]

Lista cerrada (4): (0,0) (1,0) (0,1) (2,0) · **Creados:** 6 · **Expandidos:** 4

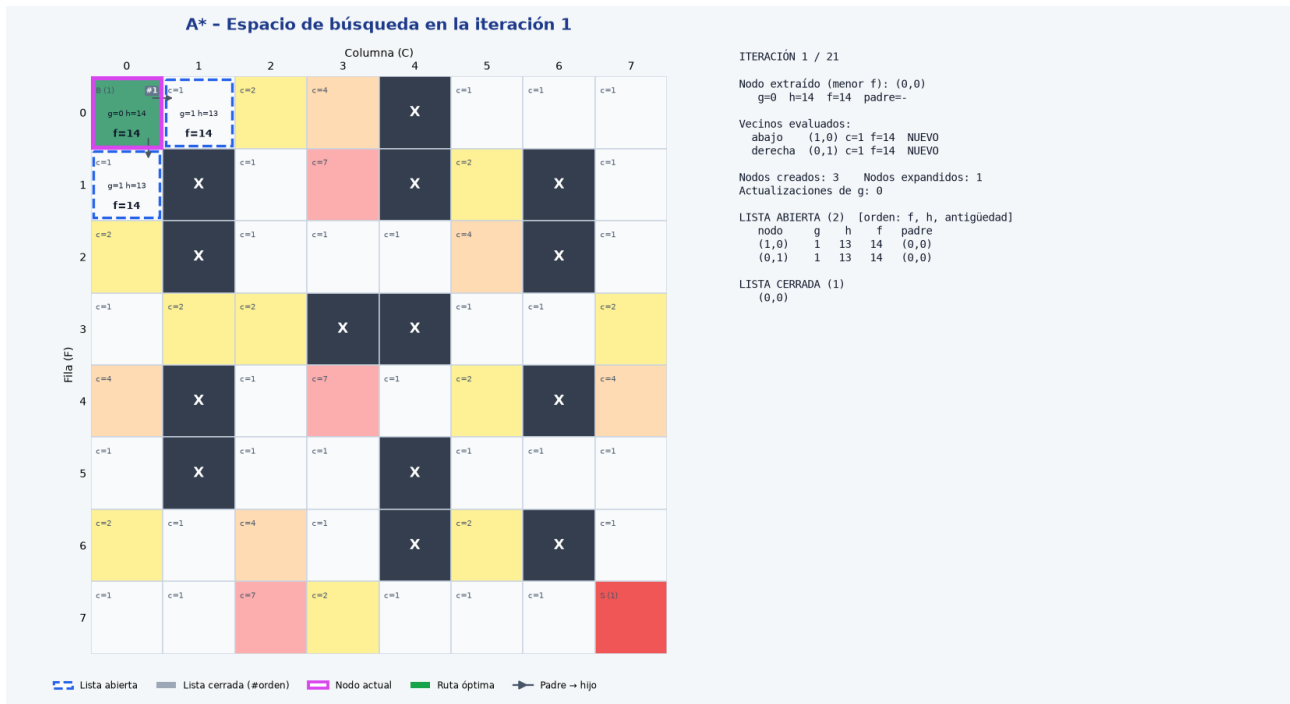


Figura 2. Estado del algoritmo en la iteración 1 (tablero + variables).

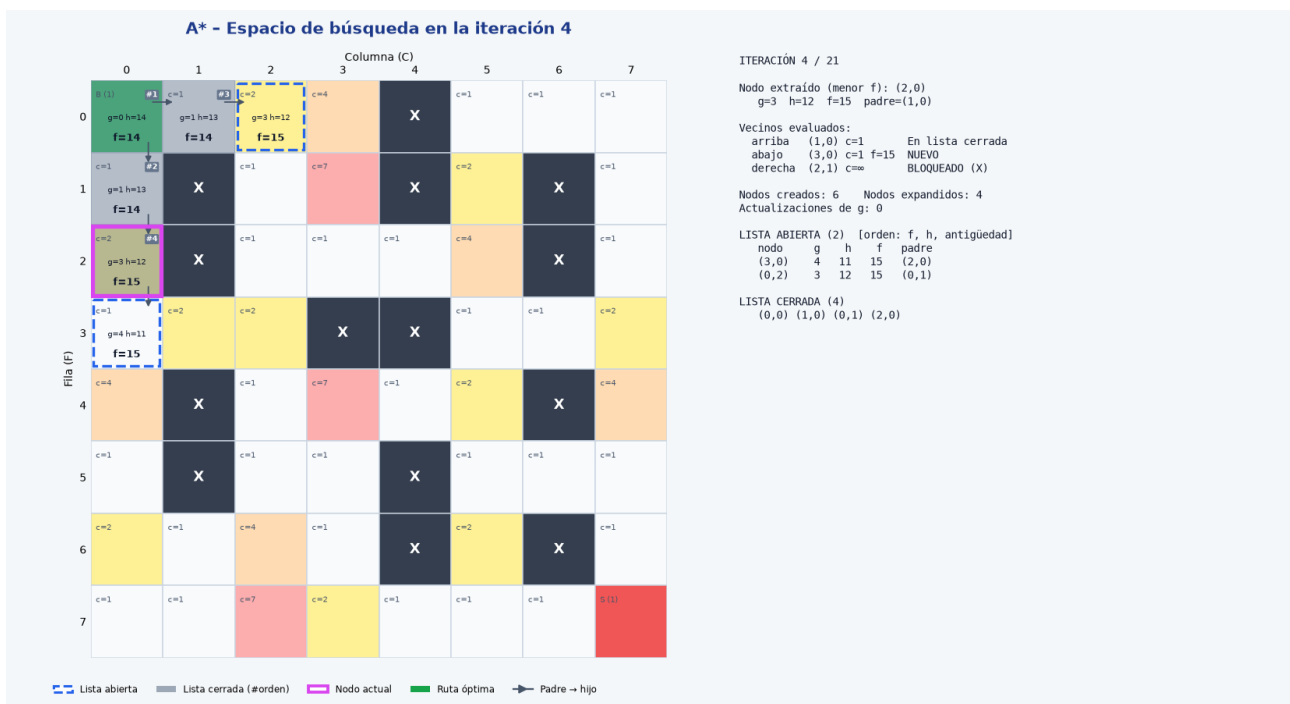


Figura 3. Estado del algoritmo en la iteración 4 (tablero + variables).

6.1 Tabla resumen de todas las iteraciones

Para cada iteración: nodo extraído (el de menor f en la lista abierta), sus valores, los vecinos nuevos que generó, el tamaño de las listas y los contadores acumulados.

It.	Nodo extraído	g	h	f	Vecinos generados / actualizados	Abierta	Cerrada	Creados	Expandidos
0	– (inicio)	–	–	–	(0,0) insertado	1	0	1	0
1	(0,0)	0	14	14	(1,0), (0,1)	2	1	3	1
2	(1,0)	1	13	14	(2,0)	2	2	4	2
3	(0,1)	1	13	14	(0,2)	2	3	5	3
4	(2,0)	3	12	15	(3,0)	2	4	6	4
5	(3,0)	4	11	15	(4,0), (3,1)	3	5	8	5
6	(0,2)	3	12	15	(1,2), (0,3)	4	6	10	6
7	(1,2)	4	11	15	(2,2), (1,3)	5	7	12	7
8	(2,2)	5	10	15	(3,2), (2,3)	6	8	14	8
9	(2,3)	6	9	15	(2,4)	6	9	15	9
10	(2,4)	7	8	15	(2,5)	6	10	16	10
11	(3,2)	7	9	16	(4,2)	6	11	17	11
12	(4,2)	8	8	16	(5,2), (4,3)	7	12	19	12
13	(5,2)	9	7	16	(6,2), (5,3)	8	13	21	13
14	(5,3)	10	6	16	(6,3)	8	14	22	14
15	(6,3)	11	5	16	(7,3)	8	15	23	15
16	(3,1)	6	10	16	ninguno	7	16	23	16
17	(7,3)	13	4	17	(7,2), (7,4)	8	17	25	17
18	(7,4)	14	3	17	(7,5)	8	18	26	18
19	(7,5)	15	2	17	(6,5), (7,6)	9	19	28	19
20	(7,6)	16	1	17	(7,7)	9	20	29	20
21	(7,7) META	17	0	17	fin	8	21	29	20

* = vecino que ya estaba en la lista abierta y cuyo $g(n)$ se mejoró (actualización).

7. Solución: ruta óptima y espacio de búsqueda

7.1 Ruta óptima

Ruta encontrada: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (6,3) \rightarrow (7,3) \rightarrow (7,4) \rightarrow (7,5) \rightarrow (7,6) \rightarrow (7,7)$

Coste total $g(7,7) = 17 \cdot 14 \text{ movimientos} \cdot 21 \text{ iteraciones} \cdot 29 \text{ nodos creados} \cdot 20 \text{ nodos expandidos} \cdot 0 \text{ actualización(es) de } g$.

Paso	Casilla	Tipo de terreno	Coste	g acumulado	h	f = g + h
0	(0,0)	Bombero (inicio)	0	0	14	14
1	(0,1)	Libre	1	1	13	14
2	(0,2)	Leve - Escombros Menores	2	3	12	15
3	(1,2)	Libre	1	4	11	15
4	(2,2)	Libre	1	5	10	15
5	(3,2)	Leve - Escombros Menores	2	7	9	16
6	(4,2)	Libre	1	8	8	16
7	(5,2)	Libre	1	9	7	16
8	(5,3)	Libre	1	10	6	16
9	(6,3)	Libre	1	11	5	16
10	(7,3)	Leve - Escombros Menores	2	13	4	17
11	(7,4)	Libre	1	14	3	17
12	(7,5)	Libre	1	15	2	17
13	(7,6)	Libre	1	16	1	17
14	(7,7)	Superviviente (meta)	1	17	0	17

Interpretación (Paso 3 del enunciado). La ruta baja por la columna 2 y luego avanza por la fila 7. El algoritmo decidió **atravesar** tres casillas de escombros leves (coste 2) en (0,2), (3,2) y (7,3) porque rodearlas resultaba más caro, y **evitó** las casillas de coste 4 y 7 cercanas al camino, como (0,3), (1,3), (4,3), (6,2) y (7,2), rodeándolas por vías libres. La mejor alternativa por el lado derecho del tablero, $\dots \rightarrow (2,4) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5) \rightarrow (5,6) \rightarrow (5,7) \rightarrow (6,7) \rightarrow (7,7)$, obliga a pasar por (2,5) con coste 4 y (4,5) con coste 2 y cuesta 19, por eso fue descartada. El último paso, entrar a S (7,7), suma 1 porque la casilla del Superviviente vale 1.

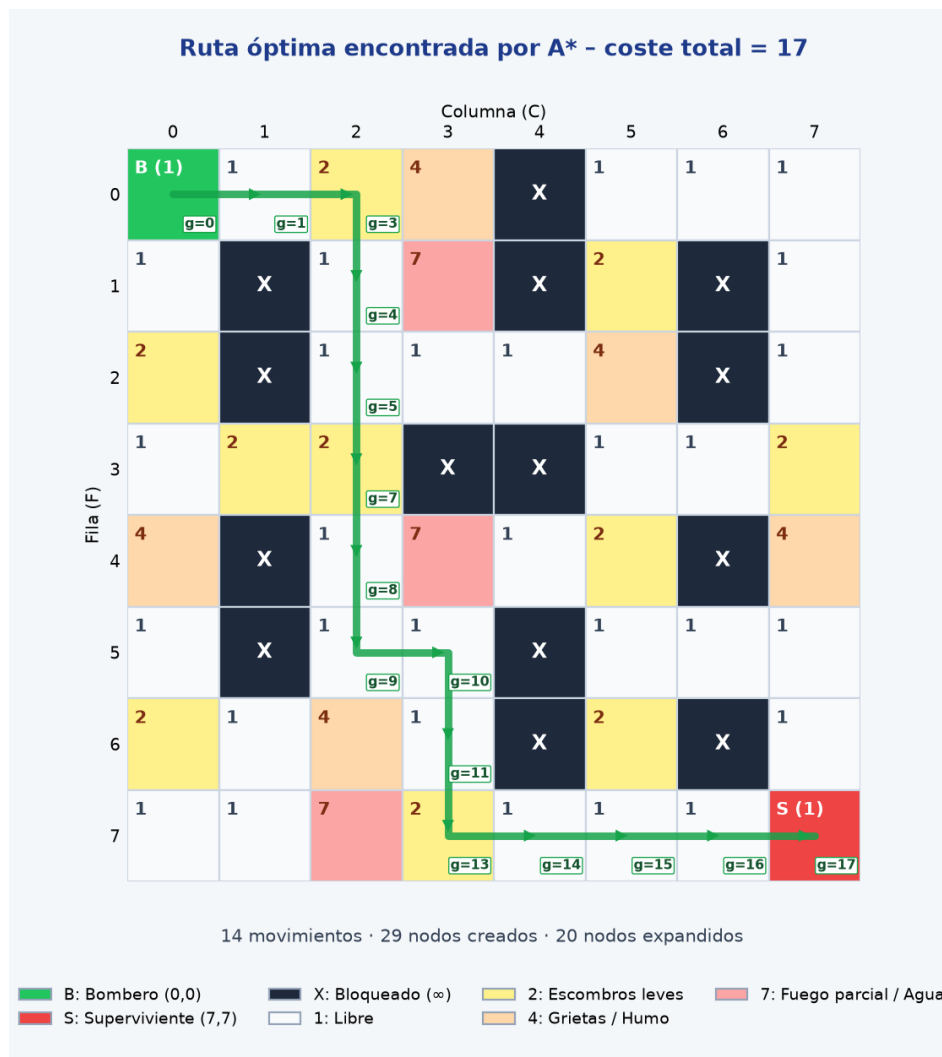
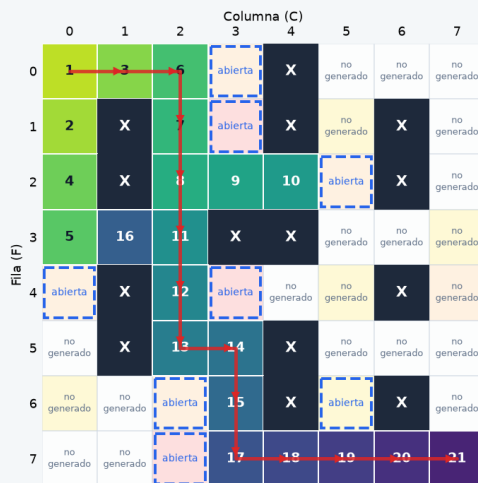


Figura 4. Ruta óptima sobre el tablero, con el g acumulado en cada casilla.

7.2 Visualización del espacio de búsqueda

La siguiente figura muestra **cómo exploró A***: cada casilla lleva el número de orden en que fue extraída de la lista abierta (entró en la lista cerrada). Las casillas con recuadro azul quedaron generadas en la lista abierta sin llegar a expandirse, y las marcadas "no generado" nunca fueron alcanzadas por la búsqueda.

A* con h = Manhattan
coste=17, expandidos=20, creados=29



Sin heurística (h = 0, coste uniforme)
coste=17, expandidos=46, creados=50

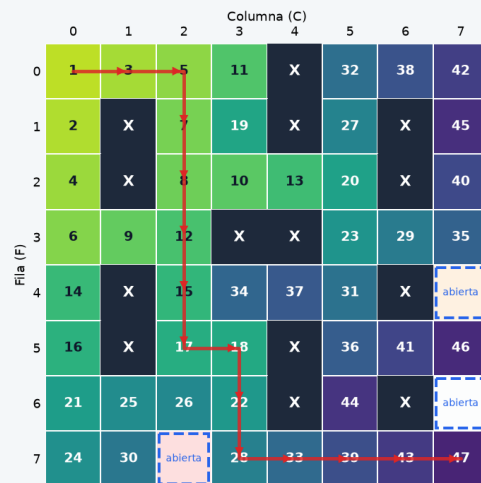


Figura 6. Orden de expansión con Manhattan (izquierda) y sin heurística (derecha).

La mejora es clara: sin heurística se expanden 46 nodos, prácticamente todo el tablero, mientras que A* con Manhattan solo expande 20. Como se vio en la Sección 5.5, la heurística está muy cerca del coste real sin superarlo nunca, así que orienta la búsqueda hacia el superviviente y evita expandir casillas que no pueden formar parte de una ruta mejor.

8. Implementación en Python y visualización

El proyecto está implementado en Python 3 con las bibliotecas matplotlib (gráficos y visor interactivo), Pillow (animación GIF) y reportlab (este informe).

Archivo	Contenido
astar_rescate.py	Matriz 8×8 del enunciado, costes, Distancia de Manhattan, algoritmo A* con lista abierta y cerrada, reconstrucción del camino, registro de la traza por iteración y verificación de admisibilidad/consistencia.
interfaz.py	Interfaz gráfica animada (Tkinter) para recorrer la búsqueda paso a paso, con animación en cada celda y todas las variables del algoritmo.
visualizacion.py	Dibujo del tablero, ruta óptima, espacio de búsqueda, imagen por iteración, animación GIF y visor clásico (matplotlib).
generar_informe.py	Genera este informe PDF a partir de los resultados reales.
main.py	Programa principal: ejecuta todo y abre la interfaz animada.

Ejecución:

```
pip install matplotlib pillow reportlab
python main.py                # todo + interfaz animada
python main.py --sin-visor    # solo genera resultados e informe
python main.py --solo-visor   # solo abre la interfaz animada
```

Resultados generados en la carpeta resultados/:

- traza_astar.txt: traza completa en texto (cada iteración con el nodo extraído, la evaluación de sus vecinos, lista abierta, lista cerrada y contadores).
- 01_tablero_inicial.png, 02_ruta_optima.png, 03_espacio_busqueda.png, 04_comparacion_h0.png, 05_admisibilidad_heuristica.png.
- iteraciones/iteracion_XX.png: una imagen por iteración con el tablero y todas las variables importantes.
- animacion_astar.gif: animación de la búsqueda completa.

Interfaz animada. Muestra a la izquierda el tablero y a la derecha las variables del algoritmo. En cada iteración: (1) un selector magenta se desliza hasta el nodo de menor f extraído de la lista abierta; (2) la celda cambia de color al entrar en la lista cerrada y recibe su número de orden $#k$; (3) se lanzan haces hacia los 4 vecinos y cada celda reacciona según el resultado: azul = nuevo nodo creado en la lista abierta, ámbar = mejora de g , rojo = casilla bloqueada X , gris = ya estaba en la lista cerrada. Los contadores de nodos creados y expandidos, el tamaño de ambas listas, la tabla de la lista abierta ordenada por f y la lista cerrada se actualizan con animación. Al extraer la meta, el bombero recorre la ruta óptima y las celdas se tiñen de verde. Se controla con botones, una línea de tiempo, velocidad ajustable y teclado ($\leftarrow$ $\rightarrow$, espacio, Inicio, Fin).

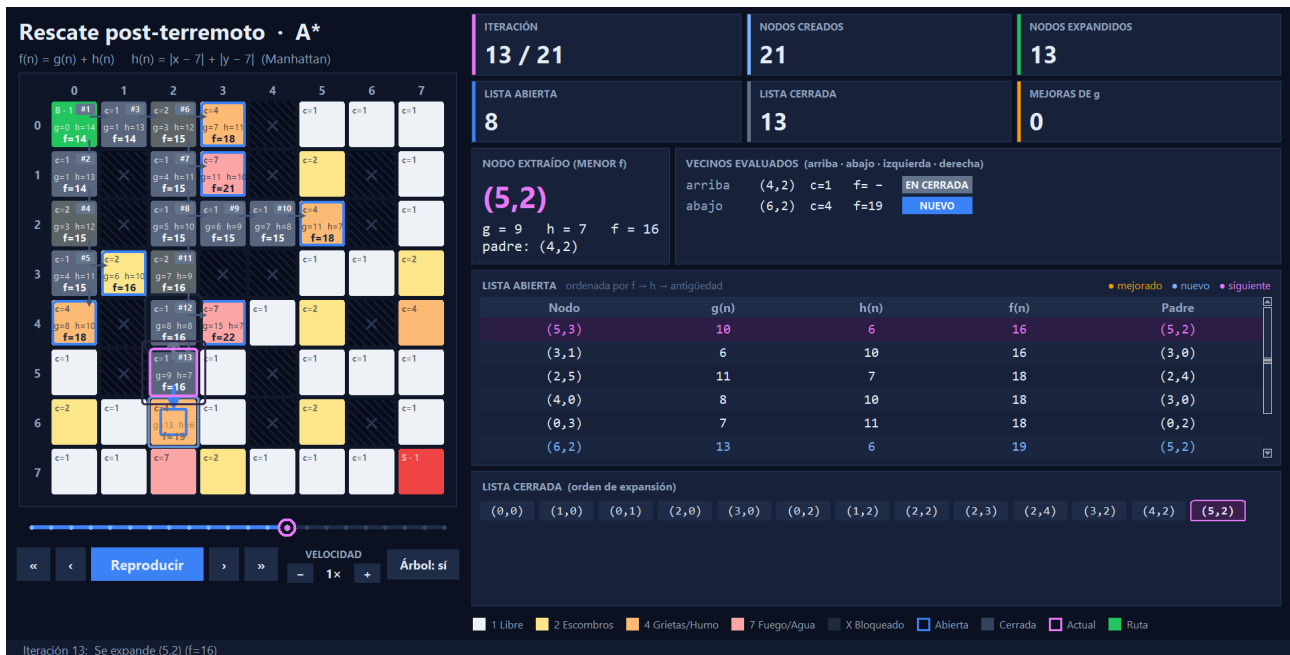
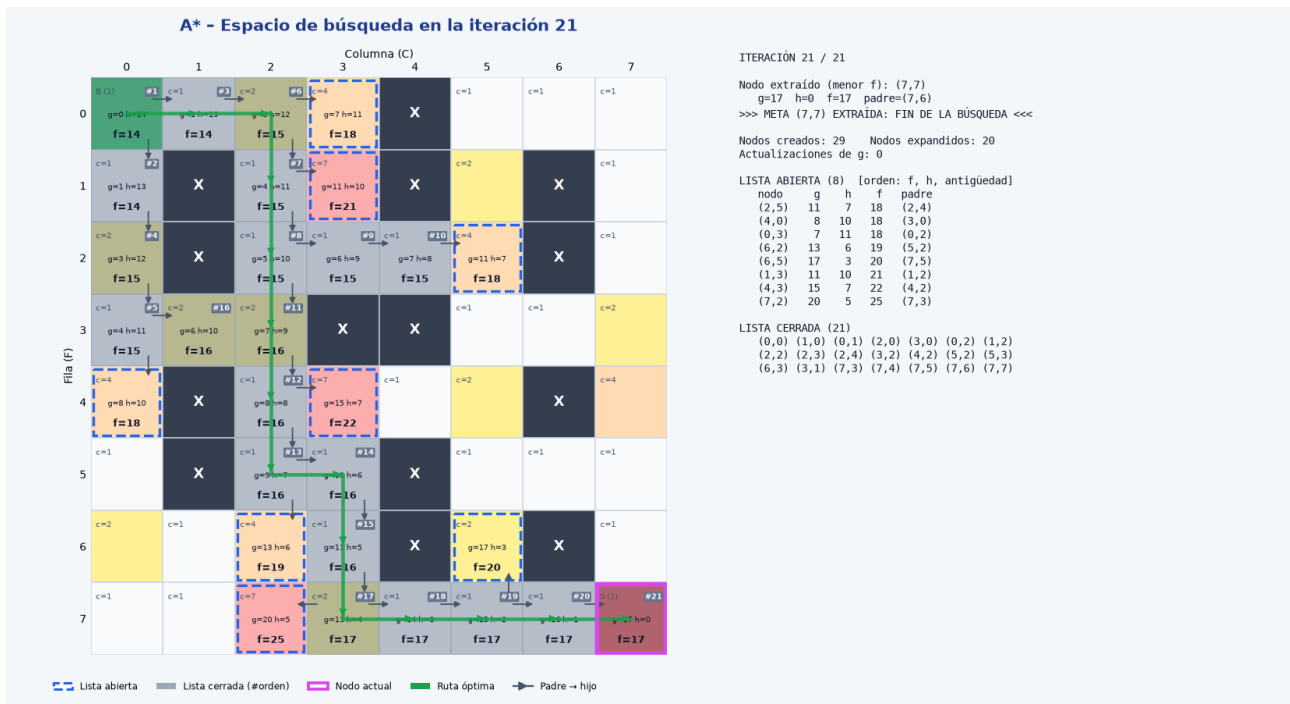


Figura 7. Interfaz animada durante la iteración 13: evaluación de los vecinos de (5,2) (gris = ya en lista cerrada, azul = nuevo en lista abierta) y listas abierta y cerrada.



- La visualización de la lista abierta, la lista cerrada y los contadores permite seguir cada decisión del algoritmo y comprobar que siempre elige el nodo de menor $f(n)$.

Anexo A. Traza completa de la lista abierta y la lista cerrada

Lista abierta al terminar cada iteración, en el orden en que se extraerán (formato (F,C):f). La lista cerrada crece un nodo por iteración, en el orden indicado en la columna "Entra a cerrada". La traza completa con g, h y padre de cada nodo está en resultados/traza_astar.txt.

It.	Entra a cerrada	Lista abierta (F,C):f	A	C
0	–	(0,0):14	1	0
1	(0,0)	(1,0):14, (0,1):14	2	1
2	(1,0)	(0,1):14, (2,0):15	2	2
3	(0,1)	(2,0):15, (0,2):15	2	3
4	(2,0)	(3,0):15, (0,2):15	2	4
5	(3,0)	(0,2):15, (3,1):16, (4,0):18	3	5
6	(0,2)	(1,2):15, (3,1):16, (4,0):18, (0,3):18	4	6
7	(1,2)	(2,2):15, (3,1):16, (4,0):18, (0,3):18, (1,3):21	5	7
8	(2,2)	(2,3):15, (3,2):16, (3,1):16, (4,0):18, (0,3):18, (1,3):21	6	8
9	(2,3)	(2,4):15, (3,2):16, (3,1):16, (4,0):18, (0,3):18, (1,3):21	6	9
10	(2,4)	(3,2):16, (3,1):16, (2,5):18, (4,0):18, (0,3):18, (1,3):21	6	10
11	(3,2)	(4,2):16, (3,1):16, (2,5):18, (4,0):18, (0,3):18, (1,3):21	6	11
12	(4,2)	(5,2):16, (3,1):16, (2,5):18, (4,0):18, (0,3):18, (1,3):21, (4,3):22	7	12
13	(5,2)	(5,3):16, (3,1):16, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22	8	13
14	(5,3)	(6,3):16, (3,1):16, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22	8	14
15	(6,3)	(3,1):16, (7,3):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22	8	15
16	(3,1)	(7,3):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22	7	16
17	(7,3)	(7,4):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22, (7,2):25	8	17
18	(7,4)	(7,5):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (1,3):21, (4,3):22, (7,2):25	8	18
19	(7,5)	(7,6):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (6,5):20, (1,3):21, (4,3):22, (7,2):25	9	19
20	(7,6)	(7,7):17, (2,5):18, (4,0):18, (0,3):18, (6,2):19, (6,5):20, (1,3):21, (4,3):22, (7,2):25	9	20
21	(7,7)	(2,5):18, (4,0):18, (0,3):18, (6,2):19, (6,5):20, (1,3):21, (4,3):22, (7,2):25	8	21

Lista cerrada final (orden de expansión): (0,0) → (1,0) → (0,1) → (2,0) → (3,0) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,2) → (4,2) → (5,2) → (5,3) → (6,3) → (3,1) → (7,3) → (7,4) → (7,5) → (7,6) → (7,7)